Lab 2 – Nhận diện cảm xúc khuôn mặt bằng Deep Learning

Ngày ra đề: 19/10/2026 · Giảng viên: TS. Nguyễn Đình Vinh

Thông tin chung

Sinh viên xây dựng hệ thống nhận diện cảm xúc khuôn mặt thời gian thực bằng deep learning, chạy trên máy tính với luồng video từ camera điện thoại (IP camera). Điểm Lab = 40% test case + 60% phỏng vấn.

Mục	Nội dung
Hình thức	Cá nhân
Thời gian thực hiện	1 tuần kể từ ngày giao đề
Hình thức đánh giá	Demo test case trực tiếp + phỏng vấn 3 câu vấn đáp
Thời lượng đánh giá	15–20 phút/sinh viên
Ngôn ngữ	Python 3.9+
Thư viện gợi ý	PyTorch hoặc TensorFlow/Keras, OpenCV, NumPy, scikit-learn, matplotlib
Thiết bị	Laptop cá nhân + điện thoại Android/iOS cùng mạng Wi-Fi

Mục tiêu

Hoàn thành Lab 1, sinh viên có khả năng:

1. Xây dựng pipeline thị giác máy tính hoàn chỉnh: thu ảnh → phát hiện khuôn mặt → tiền xử lý → phân loại cảm xúc → hiển thị kết quả.
2. Huấn luyện và đánh giá ít nhất hai mô hình deep learning (CNN tự thiết kế và mô hình transfer learning) cho bài toán phân loại ảnh.
3. Phân tích kết quả bằng accuracy, macro F1-score và confusion matrix; giải thích được lỗi của mô hình.
4. Kết nối và xử lý luồng video từ IP camera của điện thoại theo thời gian thực.
5. Trình bày, bảo vệ và giải thích các lựa chọn kỹ thuật của chính mình.

Mô tả bài toán và yêu cầu kỹ thuật

Đầu vào là luồng video từ camera điện thoại; đầu ra là khung hình có bounding box quanh mỗi khuôn mặt kèm nhãn cảm xúc và độ tin cậy (confidence), cập nhật theo thời gian thực.

Dữ liệu

- Bộ dữ liệu chính: **FER-2013** (ảnh xám 48×48, 7 lớp: Angry, Disgust, Fear, Happy, Sad, Surprise, Neutral).
- Được phép dùng thêm FER+, RAF-DB, AffectNet hoặc dữ liệu tự thu, nhưng phải ghi rõ nguồn và số lượng ảnh.
- Phải thống kê phân bố lớp và có biện pháp xử lý mất cân bằng (class weight, oversampling hoặc augmentation có chủ đích).

Yêu cầu mô hình

Mã	Yêu cầu	Tối thiểu
M1	CNN tự thiết kế (Conv–BatchNorm–ReLU–Pooling–Dropout)	Accuracy ≥ 60% trên tập test FER-2013
M2	Transfer learning (VGG16, ResNet18/50, MobileNetV2/V3 hoặc EfficientNet)	Accuracy ≥ 65% trên tập test FER-2013
M3	So sánh M1 và M2	Bảng accuracy, macro F1, số tham số, tốc độ suy luận (FPS)
M4	Phân tích lỗi	Confusion matrix + ít nhất 10 ảnh dự đoán sai kèm nhận xét
M5	Đồ thị huấn luyện	Loss và accuracy của train/validation theo epoch

Yêu cầu ứng dụng realtime

1. Đọc luồng video từ IP camera điện thoại bằng OpenCV (`cv2.VideoCapture(url)`).
2. Phát hiện khuôn mặt bằng Haar Cascade, DNN face detector của OpenCV, MTCNN hoặc MediaPipe.
3. Tiền xử lý vùng mặt đúng như lúc huấn luyện (grayscale/RGB, resize, chuẩn hoá).
4. Dự đoán cảm xúc cho **nhiều khuôn mặt** trong cùng khung hình.
5. Hiển thị bounding box, nhãn cảm xúc, confidence và FPS lên khung hình.
6. Cho phép chọn mô hình (M1 hoặc M2) qua tham số dòng lệnh, ví dụ `--model cnn`.
7. Có chế độ chạy trên **ảnh tĩnh** và **file video** ngoài chế độ camera, ví dụ `--source image.jpg`.
8. Xử lý được lỗi mất kết nối camera: thông báo rõ ràng, không crash chương trình.
9. Đạt tối thiểu 10 FPS trên laptop của sinh viên (CPU).

Kết nối máy tính với IP camera điện thoại

Điện thoại phát luồng video qua Wi-Fi; laptop đọc luồng đó bằng URL. Hai thiết bị phải cùng một mạng (hoặc laptop bắt hotspot của điện thoại).

1. Cài ứng dụng IP camera: **IP Webcam** (Android) hoặc **DroidCam** (Android/iOS), **iVCam**, **EpocCam** (iOS).
2. Mở ứng dụng, chọn *Start server*; ghi lại địa chỉ hiển thị, ví dụ `http://192.168.1.15:8080`.
3. Kiểm tra trên trình duyệt của laptop rằng địa chỉ trên xem được video.
4. Đọc luồng trong Python:

```
import cv2

URL = "http://192.168.1.15:8080/video"   # IP Webcam
# URL = "http://192.168.1.15:4747/video" # DroidCam
cap = cv2.VideoCapture(URL)

while True:
    ok, frame = cap.read()
    if not ok:
        print("Mất kết nối camera, đang thử lại...")
        cap.release()
        cap = cv2.VideoCapture(URL)
        continue
    # TODO: phát hiện khuôn mặt + dự đoán cảm xúc
    cv2.imshow("Emotion", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

5. Địa chỉ camera phải đặt trong file cấu hình hoặc tham số dòng lệnh (`--source http://...`), **không hard-code**, vì IP sẽ thay đổi khi đổi mạng lúc chấm.

Mẹo giảm độ trễ: giảm độ phân giải trên ứng dụng xuống 640×480, dùng `cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)`, hoặc đọc khung hình bằng một thread riêng.

Sản phẩm nộp

Nộp một file `MSSV_HoTen_Lab1.zip` (hoặc link GitHub công khai) trước buổi đánh giá, gồm:

```
MSSV_HoTen_Lab1/
├── README.md            # cách cài đặt, cách chạy, kết quả chính
├── requirements.txt
├── config.yaml          # URL camera, đường dẫn model, ngưỡng confidence
├── notebooks/
│   └── train.ipynb      # huấn luyện + đánh giá M1, M2 (đã chạy, còn output)
├── src/
│   ├── model.py         # định nghĩa mô hình
│   ├── detector.py      # phát hiện khuôn mặt
│   ├── predict.py       # hàm dự đoán cho 1 ảnh khuôn mặt
```

```
│   └── app.py           # ứng dụng realtime (--source, --model)
├── weights/             # trọng số đã huấn luyện (hoặc link tải)
└── report.pdf           # báo cáo 4–6 trang
```

Báo cáo gồm: mô tả dữ liệu và tiền xử lý, kiến trúc M1 và M2, siêu tham số, bảng so sánh M3, confusion matrix và phân tích lỗi M4, ảnh chụp màn hình demo với camera điện thoại.

Lệnh chạy bắt buộc phải hoạt động:

```
python src/app.py --source http://<ip>:8080/video --model cnn
python src/app.py --source test.jpg --model transfer
```

Hình thức đánh giá

Điểm Lab 1 tính trên thang 10: test case 4,0 điểm (40%) + phỏng vấn 6,0 điểm (60%). Bộ test case **không công bố trước**, giảng viên cung cấp tại buổi phỏng vấn.

Phần	Tỷ trọng	Điểm	Nội dung
Test case	40%	4,0	Chạy chương trình trên bộ test do giảng viên cung cấp tại chỗ
Phỏng vấn	60%	6,0	3 câu vấn đáp, mỗi câu 2,0 điểm
Tổng	**100%**	**10,0**	

Trình tự buổi đánh giá (15–20 phút/sinh viên):

1. Sinh viên kết nối laptop với camera điện thoại, khởi động chương trình (tối đa 3 phút).
2. Giảng viên phát bộ test case; sinh viên chạy và giảng viên ghi nhận kết quả (khoảng 7 phút).
3. Phỏng vấn 3 câu vấn đáp (khoảng 8 phút).